

BIRNIN GOBE

DOCUMENT DE PASSATION TECHNIQUE

Analyse technique & passation au développeur senior

Lecture des exigences, implications d'architecture, stack proposée, hébergement, sécurité, localisation et organisation d'une équipe de deux développeurs.

Source de référence	Cahier des charges de la plateforme web BIRNIN Gobe
Version source	1.0 - Projet détaillé pour validation
Date de l'analyse	17 août 2026
Statut	Document de travail pour revue technique

POINT DE TRAÇABILITÉ IMPORTANT

Le cahier des charges exige une infrastructure en conteneurs (§14.1). Il ne prescrit pas Docker, Docker Compose, Kubernetes ou Podman. Docker et Docker Compose sont des recommandations techniques proposées pour répondre à cette exigence avec une équipe de deux personnes.

Légende de lecture

EXIGENCE Provenant du cahier des charges	RECOMMANDATION Choix d'architecture proposé	À CONFIRMER Décision projet/client encore ouverte
--	---	---

SOMMAIRE

Vue d'ensemble

#	Chapitre	Contenu
01	Lecture fonctionnelle	Finalité du produit, portail public, candidature, admissibilité, évaluation et jury.
02	Architecture applicative	RBAC, modularité, monolithe modulaire, stack, API et stockage.
03	Localisation	Français, haoussa, zarma, contenus statiques et dynamiques, gouvernance des traductions.
04	Infrastructure & hébergement	Hébergement partagé, VPS, ancien PC Ubuntu Server, Docker, environnements.
05	Qualité & sécurité	Tests, accessibilité, sécurité, audit, machine à états.
06	Organisation à deux	Rôles, responsabilités, Jira, branches, backlog et ADR.
07	Décisions ouvertes & risques	Questions client, risques prioritaires, phase de cadrage.
08	Synthèse de la recommandation	Architecture cible, stratégie d'environnements, prochaines décisions.

COMMENT UTILISER CE DOCUMENT

Ce document sépare volontairement les obligations du cahier des charges des choix techniques proposés. Le développeur senior est invité à challenger les recommandations avant de les figer dans un blueprint technique et des ADR.

1. Objet du document

Ce document synthétise l'analyse réalisée à partir du cahier des charges et rassemble les points à transmettre au développeur senior / responsable technique.

- Ce que nous avons compris des exigences.
- Les implications techniques identifiées pendant la revue.
- Les décisions d'architecture actuellement envisagées.
- Les options d'infrastructure et d'hébergement discutées.
- Les exigences de localisation.
- Les implications d'authentification et d'autorisation.
- La stack proposée.
- Une répartition possible du travail entre deux développeurs.
- Les questions non résolues à confirmer avant l'implémentation.
- Les risques à traiter tôt.

STATUT DU DOCUMENT

Ce document ne remplace pas le cahier des charges. Les termes « Exigence », « Recommandation » et « À confirmer » servent à distinguer ce qui est contractuel de ce qui relève de notre analyse technique.

2. Résumé exécutif

Premier constat : BIRNIN Gobe n'est pas simplement un site web de compétition. Il s'agit d'une plateforme métier complète comprenant un site public, un portail candidat, des workflows administratifs, un moteur d'évaluation, un espace jury, du reporting et une piste d'audit.

Le cycle de vie principal peut être résumé ainsi :

Information -> Compte -> Brouillon -> Soumission -> Admissibilité
-> Présélection -> Sélection finale -> Publication -> Archivage

Cette nature impose une architecture, une sécurité, une autorisation, une traçabilité et une gestion des données bien plus fortes qu'un site d'information classique.

ORIENTATION ACTUELLE

Construire un monolithe modulaire plutôt que des microservices. Répondre à l'exigence de conteneurisation avec Docker/Docker Compose comme proposition initiale. Utiliser PostgreSQL, Redis, stockage objet compatible S3 et une application Laravel avec React/TypeScript.

3. Ce que les exigences décrivent réellement

3.1 Portail public

- Accueil et présentation de la compétition.
- Contexte PIDUREM.
- Thématiques.
- Conditions d'éligibilité.
- Critères d'évaluation publiés.
- Calendrier.
- Instructions de candidature.
- FAQ et assistance.
- Actualités et ressources.
- Résultats.

Le contenu doit être administrable via un CMS permettant notamment création, prévisualisation, planification, publication/dépublication, versionnement, gestion des médias et bannières d'urgence.

IMPLICATION

Le portail public ne doit pas être conçu comme un ensemble de pages HTML statiques. Une partie importante du contenu doit être administrable sans développement.

4. Portail candidat

Le workflow candidat constitue l'un des domaines les plus importants du système. Le cahier des charges décrit un formulaire multi-étapes couvrant :

1. Éligibilité guidée.
2. Profil du candidat.
3. Structure / équipe.
4. Défi et thématique.
5. Solution.
6. Impact et viabilité.
7. Plan de mise en œuvre.
8. Pièces et déclarations.
9. Relecture et envoi.

Comportements clés à prévoir :

- Sauvegarde automatique.
- Bouton explicite « Enregistrer et continuer plus tard ».
- Navigation avant/arrière sans perte.
- Progression visible.
- Validation au fil de l'eau.
- Questions et pièces conditionnelles.
- Reprise de session.
- Tolérance aux coupures.
- Téléversements reprenables lorsque cela est techniquement possible.
- Validation serveur avant soumission.
- Verrouillage du dossier après soumission.
- Horodatage serveur.
- Identifiant de soumission non prédictible.
- Reçu téléchargeable.

Après soumission, le dossier devient en lecture seule. Toute réouverture doit être autorisée, limitée, motivée et tracée.

5. Workflow administratif

Le back-office ne se limite pas à du CRUD. Il doit couvrir la configuration de campagne et le pilotage opérationnel.

- Configuration de campagne.
- Thématiques et référentiels.
- Règles d'éligibilité.
- Formulaires et pièces.
- Grilles d'évaluation.
- Utilisateurs et rôles.
- Modèles de communication.
- Publication de contenu.
- Politiques de conservation.
- Tableaux de bord, filtres, vues enregistrées et actions en lot contrôlées.

EXIGENCE STRUCTURANTE

De nombreux paramètres métier doivent être modifiables sans changement de code : dates, tranches d'âge, zones, tailles d'équipe, thèmes, questions, pièces obligatoires, critères, poids, seuils, nombre d'évaluateurs, règles d'égalité, modèles de notification, etc.

Conséquence : séparer clairement la logique applicative de la configuration de campagne. Les valeurs encore incertaines ne doivent pas être enfouies dans le code.

6. Workflow d'admissibilité

La vérification administrative introduit une machine de statuts et plusieurs contrôles : délais, profil, complétude, validité des pièces, thème, doublons, intégrité.

Des statuts conceptuels possibles :

À contrôler -> En contrôle -> Clarification demandée -> Réponse reçue
-> Admissible / Inadmissible / Arbitrage / Retiré

GARANTIE MÉTIER

Un signalement automatique (doublon probable, incohérence, document suspect, etc.) ne doit jamais, à lui seul, exclure un candidat. La décision finale reste humaine et documentée.

7. Évaluation et présélection

- Comités ou collègues d'évaluateurs.
- Affectation des dossiers.
- Équilibrage de charge.
- Expertise et disponibilité.
- Déclarations de conflits d'intérêts.
- Confidentialité.
- Évaluations indépendantes.
- Grilles pondérées.
- Commentaires.
- Verrouillage des évaluations.
- Détection des écarts.
- Consolidation.
- Proposition de shortlist.

La grille indicative de présélection totalise 100 points. Les notes sont sur une échelle de 0 à 5 et pondérées automatiquement.

RÈGLE D'ARCHITECTURE

Le backend doit être l'autorité pour le calcul officiel des scores et des pondérations. Le frontend peut afficher des calculs, mais il ne doit jamais être la seule source de vérité.

8. Sélection finale et jury

- Sessions de jury.
- Composition et présence.
- Récusations.
- Planning des pitches.
- Supports de présentation.
- Notation individuelle.

- Quorum.
- Délibération.
- Liste d'attente.
- Décisions finales.
- Incidents.
- Procès-verbal.
- Validation / visa.
- Publication.

DISTINCTION FONDAMENTALE

Décision != Publication. Un résultat validé peut rester interne jusqu'au visa de l'autorité habilitée. Ces étapes doivent être modélisées comme des permissions et transitions distinctes.

Décision du jury -> Validation interne -> Visa / approbation -> Publication

9. RBAC - contrôle d'accès basé sur les rôles

Le système doit fournir une autorisation forte pour des acteurs aux responsabilités très différentes : super administrateur, gestionnaire, vérificateur, évaluateur, jury, observateur/auditeur, communication, support et candidat.

Principe RBAC : les permissions sont affectées à des rôles, puis les utilisateurs reçoivent un ou plusieurs rôles.

Utilisateur -> Rôle -> Permissions

Exemple d'un rôle ÉVALUATEUR :

- VIEW_ASSIGNED_APPLICATION
- CREATE_EVALUATION
- EDIT_OWN_EVALUATION
- SUBMIT_EVALUATION

Il ne doit pas disposer par défaut de MANAGE_USERS, CHANGE_CAMPAGN_RULES, PUBLISH_RESULTS ou VIEW_ALL_ID_DOCUMENTS.

IMPORTANT

RBAC ne suffit pas à lui seul. Il faut également des règles au niveau de la ressource. Un évaluateur autorisé à voir des dossiers ne doit voir que ceux qui lui sont effectivement affectés.

Rappel : Authentification = « Qui êtes-vous ? » ; Autorisation = « Qu'avez-vous le droit de faire ? ».

10. Architecture modulaire

EXIGENCE

Le cahier des charges demande un backend avec un framework maintenu, une architecture modulaire, une API REST documentée et une validation serveur systématique.

Interprétation proposée : organiser le backend autour de capacités métier cohérentes plutôt qu'autour de dossiers globaux controller/service/repository.

```

Auth / Identity
Campaign
Candidate
Application
Eligibility
Evaluation
Jury
Notification
Reporting
Audit
Storage
Content / CMS
Administration

```

Chaque module possède une responsabilité claire. Exemple : Application gère brouillons, réponses, validation, progression et soumission ; Evaluation gère affectations, critères, scores, verrouillage et consolidation.

11. Monolithe modulaire - recommandation actuelle

RECOMMANDATION

Utiliser un monolithe modulaire : une application backend principale, une base relationnelle principale, un déploiement principal, mais des frontières internes fortes entre modules.

```

BIRNIN GOBE BACKEND
├─ Auth
├─ Campaign
├─ Candidate
├─ Application
├─ Eligibility
├─ Evaluation
├─ Jury
├─ Notification
├─ Reporting
└─ Audit
    │
    └─ PostgreSQL

```

Ce choix n'empêche pas d'extraire ultérieurement un composant si la charge, la disponibilité ou l'exploitation le justifie.

12. Pourquoi ne pas partir en microservices

- Découverte de services.
- Communication réseau entre services.
- Authentification distribuée.
- Transactions distribuées.
- Plusieurs déploiements.
- Pipelines CI/CD multiples.
- Observabilité distribuée.
- Tests plus complexes.
- Charge opérationnelle nettement supérieure.

Avec seulement deux développeurs, rien dans les exigences actuellement identifiées ne justifie cette complexité dès le départ.

CONCLUSION

Conserver des frontières de modules fortes, mais rester dans un seul système déployable tant que les besoins réels ne démontrent pas la nécessité de séparer.

13. Stack technologique proposée

Couche	Proposition
Architecture	Monolithe modulaire
Backend	Laravel (PHP)
Frontend public / candidat	React + TypeScript + Inertia
Back-office	Filament pour les écrans adaptés au CRUD/configuration, écrans dédiés pour les workflows complexes
CSS / UI	Tailwind CSS + composants réutilisables
Base de données	PostgreSQL
Cache / file de tâches	Redis
Traitements asynchrones	Workers Laravel / Horizon
API	REST versionnée + OpenAPI
Fichiers	Stockage objet compatible S3
Sécurité fichiers	Validation MIME + quarantaine + antivirus
Reverse proxy	Caddy (Nginx reste une alternative valable)
Conteneurs	Docker + Docker Compose - recommandation, pas exigence nominative
Tests E2E	Playwright
CI/CD	GitHub Actions
Gestion projet	Jira Free
Code source	GitHub

À CHALLENGER PAR LE SENIOR

La stack est une recommandation, pas une contrainte du cahier des charges. Elle vise surtout à réduire la plomberie et maximiser la capacité de livraison d'une équipe de deux personnes.

14. Backend Laravel - raison du choix

Laravel est proposé parce qu'il fournit dans un même écosystème de nombreux besoins du projet : routing, validation, authentification, autorisation, migrations, ORM, localisation, notifications, email, files de tâches, tâches planifiées, cache, rate limiting et abstraction de stockage.

L'objectif n'est pas de prétendre que PHP est « meilleur » que Java, Node ou Python ; il s'agit d'optimiser le rapport capacités / effort de développement pour une petite équipe.

15. Frontend React + TypeScript + Inertia

La partie candidat mérite une interface personnalisée et soignée : mobile-first, faible débit, autosave, formulaires longs, états de sauvegarde clairs, reprise de session et navigation fluide.

Inertia est proposé afin d'utiliser React sans imposer deux applications totalement indépendantes juste pour le frontend interne. Cela réduit CORS, duplication de routing, complexité d'authentification API et divergence entre deux modèles de données.

PRINCIPE

Une seule logique métier doit servir l'interface web et l'API REST. Inertia ne doit pas remplacer l'API externe demandée par le cahier des charges.

16. Back-office Filament

Filament est proposé pour accélérer les écrans administratifs standard : campagnes, thèmes, régions, communes, paramètres d'éligibilité, utilisateurs, rôles, FAQ, modèles de notification, référentiels et contenus simples.

En revanche, les écrans métier riches - vérification complète d'un dossier, comparaison des pièces, notation, session jury, délibération - devraient recevoir des interfaces dédiées plutôt que d'être forcés dans un CRUD générique.

17. PostgreSQL

Le domaine est fortement relationnel : candidat, équipe, membres, candidature, réponses, pièces, vérifications, clarifications, affectations, évaluations, notes, décisions, notifications, audit.

PostgreSQL est recommandé comme base principale. JSON/JSONB peut être utile pour certains paramètres flexibles, snapshots ou métadonnées, mais ne doit pas transformer la candidature entière en un unique blob JSON.

18. Redis et traitements asynchrones

Les opérations longues ne doivent pas bloquer les requêtes HTTP des candidats. Exemples : email, SMS, génération PDF, export, scan antivirus, recalculs ou reporting.

```
Soumission HTTP
|
+--> Enregistrer la soumission immédiatement
+--> File Redis
    | EmailJob
    | SmsJob
    | PdfJob
    | ExportJob
    | FileScanJob
```

19. API REST

EXIGENCE

Le cahier des charges demande une API REST documentée, une documentation OpenAPI, une collection de tests et une procédure d'intégration.

Proposition : conserver une couche métier commune et exposer deux façades lorsque nécessaire : contrôleurs web/Inertia et contrôleurs API /api/v1. Les deux appellent les mêmes cas d'usage pour éviter la duplication de règles métier.



20. Stockage des fichiers

Les pièces candidates peuvent inclure CV, identités, RCCM/NIF, budgets, présentations, captures, lettres et autres justificatifs.

EXIGENCE

Le cahier des charges demande un stockage objet compatible S3 ou équivalent, antivirus, somme de contrôle, URL temporaires, quotas, versions et sauvegardes.

RECOMMANDATION

Ne pas stocker les fichiers dans PostgreSQL et ne pas considérer le disque du VPS comme stockage documentaire permanent. Utiliser un stockage objet externe et ne garder en base que les métadonnées et clés de stockage.

Upload -> QUARANTAINE -> Scan antivirus -> CLEAN / REJECTED

```

attachment
- id
- application_id
- storage_key
- original_filename
- mime_type
- size
- checksum
- scan_status
- created_at

```

21. Localisation - langues prises en charge

EXIGENCE

Le français est obligatoire. L'architecture doit permettre le haoussa, le zarma et d'autres langues validées sans modification du code.

Dès le premier sprint, traiter les langues comme une capacité structurante et non comme une finition à ajouter à la fin.

```

fr = Français
ha = Haoussa
dje = Zarma (code ISO 639-3 couramment utilisé)

```

22. Localisation des textes statiques

Les textes créés par les développeurs doivent passer par des clés de traduction et ne pas être écrits en dur dans les composants.

```
common.save
common.cancel
application.submit
application.draft_saved
eligibility.warning
```

Les fichiers de ressources peuvent ensuite fournir la traduction française, haoussa et zarma.

23. Localisation des contenus dynamiques

Une grande partie des contenus vient des administrateurs : pages, FAQ, thèmes, critères, questions, textes d'aide, modèles de notification. Ces traductions doivent donc être stockées dans la base ou un CMS, pas uniquement dans des fichiers du dépôt.

```
Theme
- id
- code
- active

ThemeTranslation
- theme_id
- locale
- name
- description
```

Le même modèle peut s'appliquer à d'autres contenus multilingues.

24. Langue préférée du candidat

Proposition : enregistrer une préférence de langue dans le compte candidat, par exemple `preferred_locale`. Cette préférence peut contrôler l'interface, les emails/SMS et, si validé, certains PDF ou reçus.

FALLBACK

Prévoir un repli vers le français si une traduction haoussa ou zarma est absente, afin qu'une traduction incomplète ne rende pas l'application inutilisable.

25. Valeurs métier indépendantes de la langue

Les statuts et règles internes doivent être des codes stables, jamais des libellés français.

```
Mauvais : status = "Soumis"
Bon      : status = SUBMITTED

UI fr   : SUBMITTED -> Soumis
UI ha   : SUBMITTED -> [traduction]
UI dje  : SUBMITTED -> [traduction]
```

Même principe pour les erreurs : retourner des codes tels que INVALID_EMAIL plutôt qu'un message français figé dans la logique backend.

26. Gouvernance des traductions

Le support technique des langues et la qualité linguistique sont deux sujets différents. Les traductions sensibles doivent être relues par des personnes compétentes et validées institutionnellement.

- Éligibilité.
- Consentement.
- Confidentialité.
- Mentions légales.
- Critères et règles de sélection.
- Messages de décision.

POINT DE VIGILANCE

Une traduction machine ou générée par IA ne doit pas devenir automatiquement le texte public de référence pour des règles ayant un impact sur la candidature.

27. Analyse de l'hébergement partagé

Une offre d'hébergement partagé d'environ 40 000 CFA/an a été discutée : un site, environ 10 Go, comptes email et panneau de contrôle.

Conclusion actuelle : appropriée pour un site vitrine, une démo UI ou un aperçu client, mais pas retenue comme cible de production pour la plateforme complète tant que ses capacités techniques ne sont pas démontrées.

- Docker / conteneurs ?
- Processus persistants ?
- Redis ?
- Workers ?
- PostgreSQL ?
- Logiciels arbitraires ?
- Antivirus ?
- Supervision ?
- Accès SSH/root ?
- Limites CPU/RAM/exécution ?

28. Analyse du VPS

Une offre VPS d'environ 2 cœurs CPU, 4 Go de RAM et 165 Go SSD a également été discutée.

Elle paraît beaucoup plus adaptée, sous réserve qu'il s'agisse d'une vraie VM Linux avec SSH, sudo/root, Docker, contrôle du firewall, réseau public et workloads persistants.

CAPACITÉ

4 Go peuvent suffire pour un démarrage, mais l'OS, l'application, PostgreSQL, Redis, workers, proxy, supervision et antivirus réduisent rapidement la marge. 8 Go de RAM donneraient un environnement de production initial plus confortable.

29. Ancien PC gaming sous Ubuntu Server

Un ancien PC gaming est disponible et exécute déjà Ubuntu Server LTS.

RECOMMANDATION

L'utiliser comme environnement d'intégration / staging / recette pendant le développement. Il donne plus de contrôle qu'un hébergement partagé et permet de reproduire plus fidèlement la future production.

```

Postes développeurs -> GitHub -> Ancien serveur Ubuntu
                        |
                        STAGING / RECETTE
                        |
                        Production VPS
  
```

Utiliser uniquement des données fictives / de test sur cette machine tant que la sécurité de production n'est pas mise en place.

30. Stack serveur proposée

```

Ubuntu Server
Docker / Docker Compose
Caddy
Application
PostgreSQL
Redis
Worker(s)
Scheduler
ClamAV (si retenu pour le scan local)
  
```

Le stockage objet peut être simulé localement pendant le développement puis connecté à un fournisseur S3-compatible en production.

31. Reverse proxy

Caddy est proposé pour la terminaison HTTPS, la gestion des certificats et le proxy vers l'application. Nginx reste une alternative tout à fait valide. Ce choix n'est pas structurant à ce stade.

32. Conteneurs : exigence vs choix d'outil

EXIGENCE DU CAHIER

§14.1 Infrastructure : conteneurs, environnements développement/recette/production séparés, reverse proxy, TLS, sauvegardes, supervision et infrastructure documentée.

NOTRE CHOIX PROPOSÉ

Docker + Docker Compose pour développement, staging et production initiale. Le cahier des charges ne cite pas Docker nominativement.

```

DEV      : Docker Compose sur les postes
RECETTE  : Docker Compose sur l'ancien PC
PRODUCTION: Docker Compose sur VPS
  
```

33. Kubernetes

RECOMMANDATION

Ne pas utiliser Kubernetes au démarrage. Il augmente fortement la charge d'exploitation sans bénéfice démontré pour une équipe de deux développeurs et le volume initial attendu.

La capacité à monter en charge peut être préservée par une application stateless lorsque possible, un stockage objet séparé, des workers séparables et une base correctement dimensionnée.

34. Stratégie de tests

Le cahier demande tests unitaires, intégration, bout en bout, charge, sécurité, accessibilité et restauration.

Scénario E2E critique proposé :

```
Candidat: inscription -> brouillon -> reprise -> upload -> soumission
Vérificateur: revue -> clarification
Candidat: réponse
Vérificateur: admissible
Évaluateur: affectation -> conflit/non conflit -> note -> verrouillage
Comité: consolidation -> shortlist
Jury: évaluation -> décision
Autorité: approbation -> publication
```

Playwright est proposé pour les tests navigateur et parcours bout en bout.

35. Accessibilité et performance

EXIGENCE

La cible est WCAG 2.2 niveau AA, avec navigation clavier, focus visible, libellés, contrastes, alternatives textuelles, erreurs annoncées et authentification accessible.

- Parcours utilisable à partir de 320 px.
- Tests sur terminaux Android d'entrée/milieu de gamme.
- Navigateurs récents : Chrome, Edge, Firefox, Safari.
- Pages légères et images optimisées.
- Faible débit et connectivité instable.
- Pas de vidéo lourde en autoplay.
- Pagination des listes.
- États de sauvegarde explicites.

36. Sécurité

Les exigences de sécurité sont substantielles et doivent être intégrées dès le départ.

- MFA pour le back-office.
- Hachage robuste des mots de passe.
- Verrouillage progressif.
- Gestion des sessions.
- RBAC au moindre privilège.
- Autorisation serveur sur chaque ressource.
- Référence OWASP ASVS niveau 2.
- Protection XSS/CSRF/injection.
- Surveillance des dépendances.
- Sécurisation des fichiers.

- TLS.
- Gestion et rotation des secrets.
- Chiffrement des sauvegardes.
- Logs centralisés et alertes.
- SAST/DAST et scan de dépendances.
- Test d'intrusion avant lancement.

PRINCIPE

La sécurité ne doit pas devenir un sprint final de « hardening ». Les fondations d'authentification, autorisation, audit et fichiers doivent être posées au début.

37. Journal d'audit

Le journal d'audit doit couvrir notamment connexions, consultation de pièces sensibles, exports, changements de paramètres, affectations, notes, réouvertures, décisions, publications, suppressions et consultation du journal lui-même.

```
actor
role
action
target
old_value
new_value
timestamp
technical_source
reason
```

RECOMMANDATION

Créer un mécanisme d'audit partagé très tôt. Éviter que chaque module invente son propre format de logs métier.

38. Machine à états de la candidature

Les statuts ne devraient pas être modifiés arbitrairement depuis différents contrôleurs. Le cycle de vie doit être explicite, avec transitions autorisées.

```
DRAFT
-> SUBMITTED
-> PENDING_REVIEW
    -> CLARIFICATION_REQUESTED
    -> CLARIFICATION_RECEIVED
    -> ADMISSIBLE / INADMISSIBLE
-> IN_EVALUATION
-> EVALUATED
    -> SHORTLISTED / NOT_SHORTLISTED
-> FINALIST
    -> SELECTED / WAITLISTED / NOT_SELECTED
```

NOTE

Ce schéma est illustratif. Il doit être réconcilié avec les règles finales de campagne et les statuts exacts retenus par le client.

39. Exemple : la soumission est une opération métier

Une soumission ne devrait pas se limiter à changer un champ status.

```
submitApplication()
- vérifier la propriété du dossier
- vérifier l'état courant
- charger la configuration de campagne
- valider les champs obligatoires
- valider les conditions dynamiques
- valider les pièces
- calculer la complétude
- générer le numéro de soumission
- enregistrer l'horodatage serveur
- figer la version soumise
- effectuer la transition d'état
- écrire l'audit
- mettre en file la notification
- générer le reçu
```

Cet exemple illustre pourquoi des cas d'usage explicites et des frontières de domaine seront importants.

40. Structure de l'équipe

L'équipe comporte actuellement deux développeurs, dont un développeur senior full-stack.

Le découpage recommandé n'est pas « un frontend / un backend ». Cela créerait un fort couplage entre les personnes et des goulots d'étranglement.

ORGANISATION

Attribuer des domaines métier verticaux, tout en gardant l'architecture, la sécurité et les workflows critiques comme responsabilités partagées.

41. Rôle proposé du développeur senior

Le senior peut agir comme responsable technique / gardien de l'architecture, avec une attention particulière sur :

- Architecture globale.
- Frontières de domaines.
- Fondations d'authentification.
- Patterns RBAC.
- Mécanismes de sécurité partagés.
- Conventions de base de données.
- Conventions API.
- Infrastructure d'audit.
- Architecture de déploiement.
- CI/CD.
- Revue de code.
- Revue des workflows à haut risque.

L'objectif n'est pas qu'il devienne la seule personne capable de modifier les fondations, mais qu'il établisse des garde-fous réutilisables par les deux développeurs.

42. Proposition de domaine pour le deuxième développeur

Un domaine candidat/application complet constitue une responsabilité substantielle et formatrice, à implémenter de bout en bout.

- Profil candidat.
- Équipe.
- Candidature.
- Formulaire multi-étapes.
- Brouillons.
- Autosave.
- Complétude.
- Validation.
- Intégration des pièces.
- Soumission.
- Statut candidat.
- Réponses aux clarifications.
- Dashboard candidat.

```
UI -> Contrôleur / cas d'usage -> Domaine/service -> Persistence -> Tests
```

43. Proposition de domaine pour le senior

- Configuration de campagne.
- Dashboard administratif.
- Vérification.
- Affectation des évaluateurs.
- Conflits.
- Évaluation.
- Scoring.
- Shortlist.
- Jury final.
- Publication.
- Reporting.

Ce découpage reste une proposition. Il doit être ajusté selon les forces, disponibilités et préférences réelles de chacun.

44. Responsabilités partagées

- Architecture.
- Modèle de données.
- Sécurité.
- Authentification.
- Autorisation.
- Transitions d'état.
- Workflows critiques.
- Déploiement.
- Revue des PR importantes.

Les deux développeurs doivent pouvoir expliquer de bout en bout le parcours candidat, l'admissibilité, la notation, la décision, l'audit et la sécurité.

45. Gestion de projet avec Jira

Jira Free est suffisant pour une petite équipe et peut servir à gérer un backlog d'ingénierie simple.

```

Epic
  -> Story
    -> Task / Sub-task
  -> Bug

```

Epics possibles :

- Portail public.
- Authentification & RBAC.
- Candidature.
- Gestion des fichiers.
- Administration campagne.
- Admissibilité.
- Évaluation.
- Jury final.
- Notifications.
- Reporting.
- Audit.
- Localisation.
- Infrastructure / DevOps.
- Sécurité.

```

BACKLOG -> READY -> IN PROGRESS -> CODE REVIEW -> TESTING -> DONE

```

46. Philosophie du backlog

Éviter des tickets horizontaux du type « construire le frontend », « construire le backend » ou « faire la base ». Préférer des capacités métier démontrables.

- Le candidat peut créer un compte.
- Le candidat peut créer et reprendre un brouillon.
- Le candidat peut téléverser une pièce.
- Le candidat peut soumettre.
- Le vérificateur peut demander une clarification.
- L'évaluateur peut recevoir une affectation et noter.
- Le comité peut générer une shortlist.
- Le jury peut enregistrer une décision.
- Un utilisateur habilité peut publier un résultat approuvé.

47. Workflow Git et rythme de développement

Préférer des branches courtes et des intégrations fréquentes.

```

main
├─ feat/candidate-registration
├─ feat/application-autosave
└─ feat/admin-campaign

```

Éviter des branches frontend/backend séparées pendant plusieurs mois.

Rythme hebdomadaire possible : lundi cadrage/API, mardi-jeudi développement et PR courtes, vendredi intégration, tests E2E, démo, revue technique et préparation du prochain lot.

48. Structure de dépôt proposée

```
birnin-gobe/
├─ app/
├─ resources/
├─ database/
├─ infrastructure/
├─ docs/
│   ├── architecture/
│   ├── api/
│   ├── decisions/
│   └─ diagrams/
├─ tests/
└─ README.md
```

Dans l'application, privilégier l'organisation par capacités métier plutôt qu'un unique dossier global pour tous les controllers, services et repositories.

49. ADR - Architecture Decision Records

Maintenir de petits ADR aidera la cohérence et le futur transfert au client.

- ADR-001 Monolithe modulaire.
- ADR-002 PostgreSQL.
- ADR-003 Laravel + Inertia.
- ADR-004 Stockage objet compatible S3.
- ADR-005 Redis / queues.
- ADR-006 Stratégie de localisation.
- ADR-007 Modèle RBAC.
- ADR-008 Docker Compose pour l'exploitation initiale.

Chaque ADR devrait préciser contexte, décision, alternatives considérées et conséquences.

50. Décisions à confirmer côté projet / client

Le cahier des charges identifie lui-même plusieurs paramètres à arrêter avant la configuration finale :

- Identité graphique.
- Nom de domaine.
- Dates et fuseau horaire.
- Tranche d'âge et date de référence.
- Zones / quotas.
- Montant et modalités de subvention.
- Règles d'éligibilité.
- Pièces requises.
- Nombre de dossiers par thématique.
- Composition des comités.
- Pondérations et seuils.
- Langues.
- Canaux / fournisseurs SMS et email.
- Politique de conservation.
- Hébergement.
- Hypothèses de charge.
- Niveaux de service.

DISCIPLINE

Ces éléments doivent être suivis dans un registre de décisions/hypothèses et non devinés silencieusement par l'équipe de développement.

51. Questions techniques à trancher avant l'implémentation intensive

Sujet	Questions
Domaine / données	Jusqu'où le formulaire est-il configurable ? Versionnement des formulaires ? Snapshots immuables après soumission ? Représentation des règles d'éligibilité ?
Authentification	Email, téléphone ou les deux ? OTP ? Mot de passe + OTP ? MFA candidat ? TOTP back-office ?
Autorisation	Rôles multiples ? Rôles globaux ou par campagne ? Permissions scoppées par campagne ? Expiration des accès jury ?
Fichiers	Fournisseur S3 ? Taille max ? Antivirus ? Quarantaine ? Durée des URL signées ? Rétention ?
Notifications	Fournisseurs SMS/email ? Stratégie de retry ? Validation des templates ? Quotas/coûts ?
Localisation	Haoussa/Zarma obligatoires au lancement ? Qui valide les traductions ? Les réponses candidat peuvent-elles être multilingues ?
Infrastructure	VPS final ? Stockage objet ? Backups externes ? Monitoring ? DNS/domaine ? RPO/RTO ?

52. Risques majeurs identifiés

Risque	Problème	Réponse
Règles non finalisées	Construire trop tôt avec des valeurs codées en dur.	Configuration explicite, registre de décisions, aucun hardcoding des inconnues.
Surarchitecture	Interpréter « scalable » comme microservices/Kubernetes obligatoires.	Monolithe modulaire, conteneurs, stockage externe, composants séparables plus tard.
Sous-estimation du back-office	Traiter vérification/jury comme de simples CRUD.	Les considérer comme domaines produit à part entière.
Défauts d'autorisation	Fuites entre candidats, évaluateurs, jury, support, admin.	RBAC central, politiques, tests d'autorisation, principe de refus par défaut.
Exposition des documents	Accès indu aux pièces d'identité et autres données sensibles.	Stockage objet, URL temporaires, masquage, audit, antivirus, moindre privilège.
Localisation tardive	Réécriture coûteuse si le français est codé en dur.	Interdiction des chaînes UI codées en dur dès le sprint 1.
Faible connectivité	Perte de données, formulaires inutilisables.	Autosave, reprise, payloads légers, upload adapté, états de sauvegarde explicites.
Pic de dernière minute	Saturation juste avant la clôture.	Tests de charge, workers, stockage objet, monitoring, procédure de prolongation.

53. Première phase d'ingénierie proposée

Avant de construire les fonctions visibles, les deux développeurs devraient réaliser une courte phase de cadrage technique.

1. Architecture validée.
2. Diagramme des domaines.
3. Modèle de données initial.

4. Machine à états candidat.
5. Matrice rôles/permissions.
6. Conventions API.
7. Format d'erreurs.
8. Conventions de localisation.
9. Conventions d'audit.
10. Conception du stockage de fichiers.
11. Conventions de tests.
12. Structure du dépôt.
13. Environnement Docker de développement.
14. Pipeline CI initial.
15. Backlog Jira initial.

54. Premier jalon vertical recommandé

Le premier jalon doit prouver l'architecture de base plutôt que couvrir toute la compétition.

CANDIDAT

Inscription -> Connexion -> Dashboard -> Créer un brouillon
-> Enregistrer une section -> Déconnexion -> Reconnexion -> Reprendre le brouillon

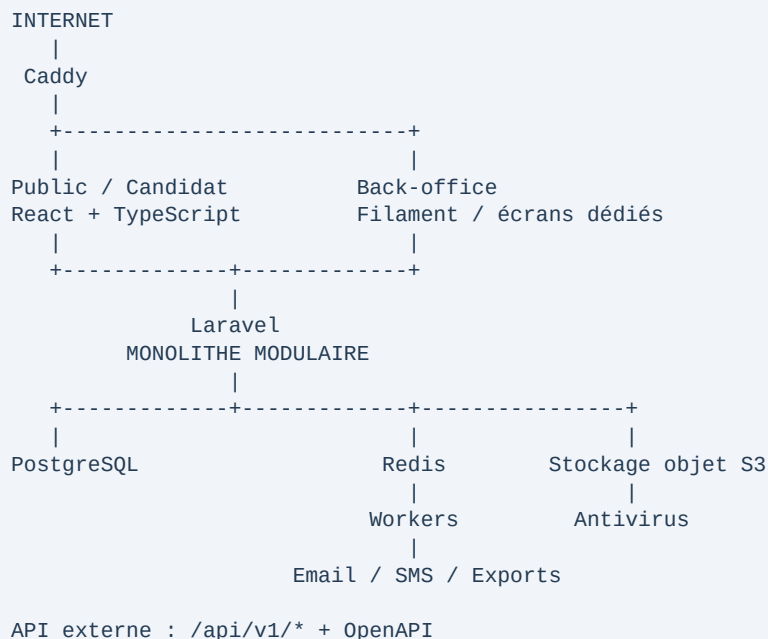
ADMIN

Connexion avec MFA -> Dashboard -> Voir le candidat / la candidature

Ce jalon valide authentification, autorisation, base de données, communication frontend/backend, propriété des données, accès admin, persistance, conteneurs, déploiement et audit de base.

ARCHITECTURE CIBLE - SYNTHÈSE

Position technique actuelle



Stratégie d'environnements :

```

DÉVELOPPEMENT
Postes des développeurs + conteneurs
    |
    v
GitHub / CI
    |
    v
RECETTE / STAGING
Ancien PC gaming - Ubuntu Server LTS - données fictives
    |
    v
PRODUCTION
VPS + stockage objet externe + sauvegardes hors serveur
  
```

POSITION GLOBALE

Construire BIRNIN Gobe comme une plateforme métier sécurisée, configurable et traçable, avec un monolithe modulaire simple à exploiter pour une équipe de deux personnes. Ne pas sacrifier l'autorisation, l'audit, la sécurité des fichiers, la localisation et l'intégrité des workflows au nom de la vitesse.

Revue technique attendue du senior

Avant de figer la stack et de démarrer l'implémentation, les cinq propositions suivantes doivent être challengées en priorité :

1. Laravel + React/TypeScript + Inertia + Filament.
2. Monolithe modulaire.
3. PostgreSQL + Redis + stockage objet compatible S3.
4. Docker/Docker Compose pour répondre à l'exigence de conteneurisation.
5. Découpage des domaines et répartition de responsabilité entre les deux développeurs.

Une fois ces points validés, l'artefact suivant devrait être un Blueprint technique BIRNIN Gobe contenant les frontières de modules, le modèle de données, les machines à états, la matrice RBAC, les conventions API, le modèle de localisation, le cycle des uploads, la topologie de déploiement, le baseline sécurité et le backlog initial.

Source principale

Cahier des charges - Conception, développement, déploiement et maintenance de la plateforme-site web de la compétition nationale « BIRNIN Gobe », version 1.0, 17 août 2026. Références particulièrement structurantes : sections 3 à 18, notamment §14.1 (architecture/infrastructure), §15 (sécurité), §16 (UX/accessibilité) et §17 (réalisation/tests).